SIMATS ENGINEERING

CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1712

Register Number	**1923**72085
Name	**M** .indu priya

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

I M.SANDHYA(192312421) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Answers :

Question 1: Emergency Drone Delivery (Greedy Best-First Search vs A*)

i. Greedy Best-First Search (GBFS) Behaviour

GBFS always expands the node whose heuristic value h(n) - the straight-line distance to the destination - is smallest, completely ignoring the cost already spent, g(n). In this scenario the drone would repeatedly head toward whichever waypoint looks geometrically closest to its target, giving very fast decisions.

- Strength: extremely fast to compute and reacts quickly - well suited to the drone's urgency requirement, since it commits to a plan with minimal computation.
- Strength: low overhead makes it attractive when onboard compute/battery is limited.
- Weakness: ignoring real movement cost means it can pick a route that looks short in a straight line but is actually slow or dangerous due to weather or terrain.
- Weakness: not optimal and not guaranteed complete - it can be misled into a path that leads toward a blocked or hazardous area simply because that direction looked closest to the destination.
- Weakness under uncertainty: since it never reconsiders cost, a route chosen early can turn out far worse once real-time blockage/weather updates arrive, and GBFS has no cost bookkeeping to fall back on when replanning.

ii. How A* Handles the Same Situation

A* expands nodes in order of f(n) = g(n) + h(n): the actual accumulated cost so far (reflecting terrain and weather penalties) plus the heuristic estimate of the remaining straight-line

distance. This balances "how far have I already had to travel/pay" against "how much further does this look like it needs to go."

- Because g(n) folds in real weather/terrain costs, a route that looks close in a straight line but is actually costly (e.g., through a storm) gets penalised appropriately, unlike in GBFS.
- As long as the straight-line-distance heuristic is admissible (never overestimates real travel cost, which it naturally is not, since straight-line distance is always ≤ real path distance), A* is guaranteed to return the least-cost, safest-by-cost route given current known information.

iii. Impact of Heuristic Accuracy on Both Algorithms

Aspect	Greedy Best-First Search	A*
Effect of an inaccurate heuristic	Directly derails the search - since GBFS has no cost-based correction, a misleading heuristic can send the drone down a poor route with no safeguard	Correctness (optimality) is preserved as long as the heuristic stays admissible; only search efficiency (number of nodes expanded) degrades
Effect of a more accurate heuristic	Faster and better routes, but still no optimality guarantee even with a good heuristic	Fewer nodes need to be expanded, so the search becomes both faster and remains optimal

In short: heuristic accuracy is essential for both speed and correctness in GBFS, but for A* it mainly affects speed - correctness is protected as long as admissibility holds.

iv. Impact of Dynamic Changes (Blocked Paths) on Search Performance

- Both algorithms assume a largely known graph at planning time; if a road becomes blocked after a route is chosen, the previously computed path can become invalid mid-flight.
- GBFS is especially vulnerable: it commits quickly to a greedy-looking path and keeps no useful cost bookkeeping, so discovering a blockage late forces an almost total restart of the greedy decision process.
- A* is better positioned because it maintains cost-so-far information for the whole frontier, so re-planning can reuse some of that computation rather than starting from scratch - though plain A* still needs to be re-invoked whenever new blockage information arrives; true responsiveness benefits from pairing A* with an incremental/online replanning technique (e.g., D* Lite).

v. Recommendation

A* is the more suitable algorithm overall. Although GBFS reacts faster per decision, its lack of an optimality or safety guarantee is dangerous when the cargo is emergency medicine - choosing a shorter-looking but actually blocked or high-risk route could cost critical time or the drone itself. A* balances cost and heuristic to guarantee the best currently-known route, and when paired with periodic re-planning (or an incremental variant such as D* Lite) as new blocked-path updates arrive, it also meets the real-time responsiveness the mission demands.

This combination best satisfies the scenario's three priorities: real-time constraints, optimality, and safety.

Question 2: Smart City Traffic Signal Optimisation (Local Search)

Formulating the Problem as Optimisation

Component	Definition
State	A complete assignment of signal-timing parameters (green-light duration and phase offset) for every intersection in the city
Objective function	Minimise a weighted combination of total waiting time, fuel consumption, and traffic congestion across the network
Search space	Enormous and largely continuous - hundreds of intersections, each with several tunable timing parameters, giving a combinatorial/continuous space with no natural discrete "path" structure

Why traditional (path-based) search is inefficient: algorithms like BFS, DFS, UCS, or A* are built to find a path from a start state to a discrete goal state through a graph. Here there is no single well-defined goal state, only a continuously improvable quality score, and the space is far too large and dynamic (traffic patterns shift constantly) for systematic path exploration to keep up. This calls for local/iterative-improvement search rather than path search.

i. Hill Climbing in This Scenario

Hill climbing starts from an initial timing configuration and repeatedly moves to a neighbouring configuration (e.g., nudging one intersection's green time slightly) whenever that neighbour scores better, stopping when no neighbour improves the score.

- Limitation: gets trapped at local maxima - a configuration that is best among small nearby tweaks but far from the best possible city-wide.
- Limitation: can stall on plateaus, where many neighbouring configurations score almost identically and there is no clear direction to move.
- Limitation: struggles on ridges, where the true improving direction requires several coordinated changes at once rather than a single-step move.
- Limitation: never accepts a worse move, so once trapped it cannot escape without restarting from a different initial configuration.

ii. Local Maxima, Plateaus, and Ridges - Real-World Interpretation

- Local maximum: a timing pattern that reduces congestion within one small downtown cluster of intersections, where every small tweak from there makes that cluster locally worse - even though a larger, coordinated re-timing across a wider area (including a nearby highway feeder road) would be substantially better overall.
- Plateau: adjusting a suburban intersection's green duration by a couple of seconds barely changes overall congestion at all, so hill climbing has no signal telling it which direction is actually useful.
- Ridge: getting a smooth "green wave" along a major arterial road requires changing several consecutive intersections' offsets together; changing just one at a time (a typical

hill-climbing move) may show no improvement even though the correct simultaneous, multi-intersection change would meaningfully cut congestion - much like a mountain ridge where every single-step direction looks like “down” except along the ridge itself.

iii. How Simulated Annealing / Genetic Algorithms Overcome These Limitations

- Simulated Annealing (SA): occasionally accepts a worse move, with a probability that decreases over time (the “cooling schedule”). This lets the search deliberately step downhill temporarily to escape a local maximum or cross a plateau, gradually settling into a strong solution as the acceptance of worse moves tapers off.
- Genetic Algorithms (GA): maintain a whole population of candidate timing plans at once rather than a single configuration, and evolve them through selection (favour lower-congestion plans), crossover (combine a good downtown timing pattern from one candidate with a good highway pattern from another), and mutation (randomly perturb some values to keep the search diverse). Exploring many regions of the space simultaneously makes GA far less likely to get stuck behind a single local maximum or ridge.

iv. Recommended Approach and Justification

Genetic Algorithms are the best overall fit for this scenario, given the scale (hundreds of intersections) and the constant need to adapt to changing traffic patterns:

- Scalability: GA's population-based evaluation is naturally parallelisable across many intersections/candidate plans, matching the city-wide scale of the problem.
- Adaptability: the population can be periodically re-evolved using fresh traffic data as new “generations,” keeping the timing plan responsive to changing conditions rather than fixed to a single moment's traffic pattern.
- Escaping traps: crossover lets GA recombine good partial solutions from very different regions of the city, which is exactly what is needed to escape the ridge-type traps that block single-point methods like hill climbing.

Question 3: Mars Rover Exploration (Online Search)

i. Why This Problem Requires an Online Search Agent

Offline search (standard A*/UCS) requires the full environment map to be known in advance so a complete path can be computed before any movement begins. The rover has no such map - it only discovers terrain features as its sensors reach them - so it cannot pre-compute a full route. It must interleave computation with actual movement and sensing, which is exactly what defines an online search agent. Because physical moves can be effectively irreversible (already committed once taken) and communication delay with Earth rules out constant human guidance, the rover must act on incomplete knowledge and revise its plan as it learns more.

ii. Challenges of Partial Observability and Dynamic Environments

- Partial observability: sensors only reveal a limited local radius, so hazards beyond that range (craters, rocks) stay unknown until the rover gets close, risking sudden encounters with danger.
- Possible backtracking: a chosen path may turn out to be a dead end or unsafe only after the rover has already committed resources to it.

- Dynamic environment: dust storms or shifting sand can alter terrain the rover already sensed, invalidating previously "safe" paths and forcing continuous re-evaluation.
- Combined effect: the rover cannot guarantee an optimal path in advance - its actual performance depends on how well it makes local decisions and accumulates knowledge, and in the worst case it may travel considerably more than the shortest path that would exist if the whole map were known upfront.

iii. How the Rover Builds and Updates Its Internal Model

- Maintains an internal partial map (e.g., an occupancy grid) marking each region as free, obstacle, or unknown, updated whenever new sensor data arrives.
- Fuses data from multiple sensors (camera, LIDAR, inertial measurement) to refine position estimates and detect new obstacles - similar in spirit to SLAM (Simultaneous Localisation and Mapping).
- When an unexpected obstacle is found, the model is updated immediately and a local re-plan is triggered (e.g., an incremental search such as D* or a fresh local A*/DFS over the currently known map) to find an alternate route.
- Over time the model becomes progressively more complete for regions already visited, though full global knowledge is never guaranteed unless the whole area is eventually explored.

iv. Online Search vs Uninformed vs Informed Search

Strategy	**Key characteristic**	**Fit for the rover**
Uninformed search (BFS/DFS/UCS)	No domain guidance; assumes the full graph is available up front	Poor fit alone - requires a complete map the rover does not have
Informed search (A*, Greedy Best-First)	Uses a heuristic (e.g., distance to the target sample site) but classically still assumes the graph is known offline	Useful in spirit (heuristic guidance is valuable) but needs adapting for incremental/online use
Online search (Online DFS, LRTA*)	Assumes nothing is known in advance; the agent must physically move to a location to discover its neighbours, and can combine local heuristic estimates with real exploration and backtracking	Best fit - matches the rover's need to sense, decide, and act incrementally under uncertainty

v. Justification of the Most Suitable Approach

A heuristic-guided online search agent (an LRTA*-style approach) is the most suitable choice, because it:

- Naturally handles partial observability, since it only acts on locally sensed information and updates its knowledge as it goes.
- Adapts to a dynamic environment, since it can replan as soon as new hazards or terrain changes are detected.

- Uses heuristic guidance (e.g., distance to the target sample site) to keep exploration efficient despite lacking a complete map - more efficient than blind uninformed online exploration.
- Improves safety, since the rover only commits to moves based on currently sensed knowledge and can favour paths that keep a margin from any detected hazard, rather than assuming unseen areas are safe.

Question 4: University Exam Timetabling (CSP Formulation)

i. Variables, Domains, and Constraints

Component	Definition	Real-world reasoning
Variables	One variable per exam/course to be scheduled: E1, E2, …, En	Each course's exam is an independent decision that needs a time slot and hall
Domain (per variable)	Domain(Ei) = set of valid (time slot, exam hall) pairs, e.g., {(Slot1, HallA), (Slot1, HallB), (Slot2, HallA), …}	An exam can, in principle, be placed in any available slot-hall combination unless restricted by a constraint
No-overlap constraint	No two exams that share at least one common student may be assigned the same time slot	Prevents any student from having two exams scheduled simultaneously
Hall-capacity constraint	The number of students taking an exam must not exceed the assigned hall's capacity, and only one exam per hall per slot	Physical rooms have fixed seating limits and cannot host two exams at once
Precedence constraint	Certain subjects must be scheduled before others	Reflects academic sequencing, e.g., a prerequisite theory paper before its follow-on practical/viva
Faculty-availability constraint	The invigilating faculty member for an exam must be free (not already assigned elsewhere) at that slot	A faculty member cannot invigilate two exams at the same time
Special-accommodation constraint	Certain students require extended time or a separate quiet hall, so their exam(s) must be assigned hall/slot types meeting those accessibility needs	Ensures fairness and compliance with accessibility requirements

ii. How Backtracking Search Works in This Scenario

Exams are assigned to (slot, hall) values one at a time - typically choosing the most constrained exam first (e.g., the course with the most conflicting students). After each

assignment, every relevant constraint (no student overlap, hall capacity, precedence, faculty availability, accommodations) is checked. If a violation occurs, that value is rejected and the next candidate value is tried; if no value works for the current exam, the algorithm backtracks to the previous exam and tries a different assignment there. This continues until either a complete, conflict-free timetable is produced, or the search concludes that no solution exists under the current constraints (which may prompt the university to add halls or slots).

iii. How Constraint Propagation (Forward Checking) Improves Efficiency

After each partial assignment, forward checking looks ahead and removes now-invalid values from the domains of not-yet-assigned exams. For example, once exam E1 is fixed at (Slot 3, Hall A), any exam sharing a student with E1 immediately has Slot 3 removed from its domain, and any exam that would also need Hall A at that slot is pruned too. This shrinks the branching factor early and reveals dead ends (a variable left with an empty domain) far sooner than plain backtracking would, avoiding wasted search deep down a branch that was already doomed. Stronger techniques such as arc-consistency (AC-3) can be layered on top for even more thorough pruning across the whole constraint network, not just constraints touching the most recently assigned exam.

iv. Real-World Challenges and the Best Strategy for Scalability

- As the number of courses and students grows, the constraint graph becomes extremely dense (many pairwise student-conflict constraints), making naive backtracking too slow in practice.
- Real universities also have soft preferences (e.g., spacing exams out for students) in addition to hard constraints, which effectively turns part of the problem into constrained optimisation rather than pure satisfaction.

Question 5: Real-Time Strategy Game AI (Minimax & Alpha-Beta Pruning)

i. How the Minimax Algorithm Models This Problem

The game is represented as a tree of alternating turns: the MAX player (the AI) tries to maximise the outcome value, and the MIN player (the human opponent) tries to minimise it. The algorithm recursively evaluates future states down to some depth (or true terminal states), then backs values up from the leaves toward the root, always assuming the opponent will play optimally against the AI (worst-case reasoning). The AI then picks the root-level move leading to the branch with the best guaranteed (minimax) value.

ii. Computational Challenges of Large Game Trees

- The number of nodes grows exponentially with both the branching factor (legal moves per turn) and search depth - O(b^d) - which quickly becomes infeasible for a real-time strategy game with many possible actions per turn.
- Strict per-move time limits mean the AI cannot search to the true end of the game, so it must use a depth-limited search with a heuristic evaluation function at the cutoff, introducing approximation instead of a guaranteed-correct terminal value.
- Memory also grows with the size of the tree being explored/stored, adding further pressure alongside the time constraint.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-beta pruning tracks two bounds while traversing the tree: alpha (the best value MAX can guarantee so far) and beta (the best value MIN can guarantee so far). Whenever a node's value is found to be worse for the player to move than an already-explored alternative elsewhere in the tree (i.e., beta ≤ alpha), the remaining branches beneath that node can be safely skipped, because a rational opponent would never allow play to reach that branch anyway.

- This produces exactly the same decision as plain minimax, but explores far fewer nodes.
- With ideal move ordering (best moves considered first), the effective branching factor can drop from b to roughly $\sqrt{b}$, letting the search go about twice as deep within the same time budget - critical under the scenario's strict real-time limits.

iv. Designing an Evaluation Function

Since full-depth search is infeasible, an evaluation function estimates how good a non-terminal state is for the AI by combining weighted factors.Each factor is weighted according to its strategic importance (tuned through testing or self-play) and combined into a single numeric score, higher meaning better for the AI. These factors are chosen because together they capture both the immediate tactical picture (threats, material) and the longer-term strategic position (territory, economy) that ultimately determines the game's outcome - letting the AI make sound decisions without ever searching to the true end of the game.

v. Recommended Strategy for Balancing Optimality and Real-Time Performance

Use Alpha-Beta pruning combined with iterative deepening (progressively searching depth limits 1, 2, 3, … within the remaining time budget, always keeping a valid best move ready in case time runs out), move-ordering heuristics (trying previously good/likely-good moves first to maximise pruning), and a well-tuned heuristic evaluation function at the cutoff depth.

This combination gives the AI a strong, “good enough” near-optimal move within the strict time limit, trading perfect optimality (only achievable with an infeasible full-depth minimax search) for a practical balance of decision quality and responsiveness suited to real-time play.

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50